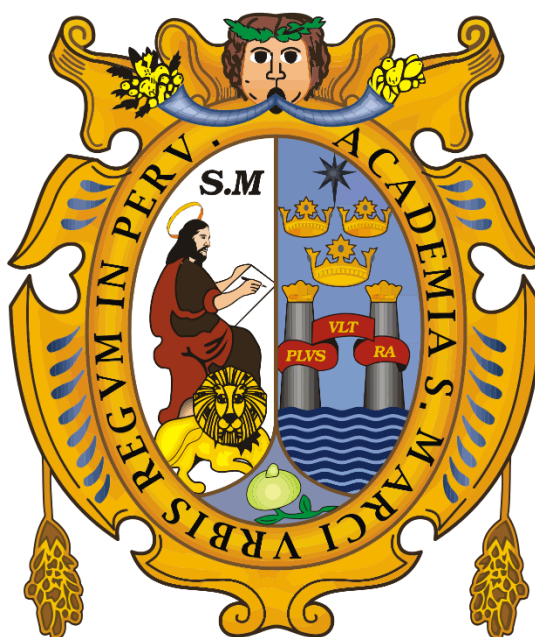


UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

(Universidad del Perú, Decana de América)

FACULTAD DE INGENIERÍA DE SISTEMAS E INFORMÁTICA

UNIDAD DE PREGRADO



Asignatura:

Introducción de Computadoras

Docente:

Torres Tineo, Cristhian Anthony

Integrante:

Mamani Huahualuque, Airton Abedal

Cruz Mora, Angel Josue

2026

Introducción

Este documento proporciona un análisis exhaustivo y detallado del sistema desarrollado en el lenguaje de programación Common Lisp. El software integra un menú interactivo que permite la ejecución de una calculadora científica de consola, la generación automática de tablas de multiplicar y el cálculo preciso de áreas de figuras geométricas.

El sistema está diseñado bajo el paradigma de programación modular y funcional, característico de Common Lisp. Se compone de un menú de control principal que despacha llamadas a diferentes módulos independientes. Esta estructura facilita el mantenimiento, la depuración y la distribución de tareas de desarrollo entre varios programadores.

1. Desarrollo

1.1 Gráfica representativa al diagrama de flujo:

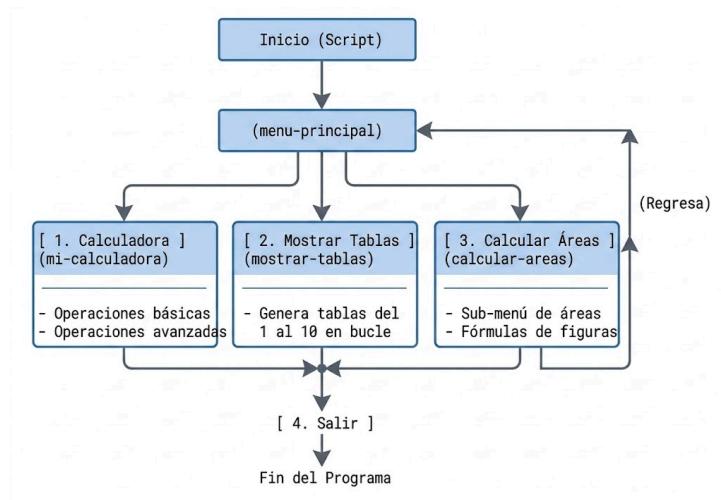


Figura 1. Representación del diagrama de flujo donde se ve resumido todo el código fuente.

1.2 Análisis detallado del código fuente de la primera parte:

1.2.1 El Menú Principal: (menu-principal)

Esta función actúa como el controlador central de la aplicación. Utiliza un bucle infinito definido por la macro (loop) y una estructura condicional (cond) para evaluar la entrada del usuario.

```
6 ; =====
7 ; MENÚ PRINCIPAL
8 ; =====
9 (defun menu-principal ()
10 "Función principal que controla el flujo del programa mediante un menú interactivo."
11 (let ((opcion 0))
12   (loop
13     (format t "~%--- MENÚ DE OPCIONES ---~%")
14     (format t "1. Ejecutar Calculadora~%")
15     (format t "2. Mostrar Tablas de Multiplicar~%")
16     (format t "3. Calcular Área de Figuras~%")
17     (format t "4. Salir~%")
18     (format t "Seleccione una opción: ")
19
20     (finish-output)
21
22     (setq opcion (read))
23
24     (cond
25       ((= opcion 1) (mi-calculadora)) ;; Llama al trabajo del Integrante 1
26       ((= opcion 2) (mostrar-tablas)) ;; Llama al trabajo del Integrante 2
27       ((= opcion 3) (calcular-areas)) ;; Llama al trabajo del Integrante 2
28       ((= opcion 4)
29         (progn
30           (format t "~%Saliendo del programa...~%")
31           (return))) ;; Rompe el bucle loop y termina
32       (t
33         (format t "~%Opción no válida, intente de nuevo.-~%"))))
34
```

Figura 2. Representación del código fuente del menú principal.

Elementos Clave Analizados:

let ((opcion 0)): Inicializa una variable local **opcion** con valor **0** para almacenar la entrada del usuario de manera segura dentro de la función.

loop: Define una iteración infinita. El bucle solo se interrumpe cuando la estructura condicional ejecuta la función de escape (**return**) al seleccionar la opción **4**.

finish-output: Esta llamada es crítica. En muchos entornos e intérpretes de Common Lisp (como SBCL o CLISP), la salida de pantalla se almacena en búfer (buffer).

finish-output fuerza la impresión inmediata en la consola, asegurando que el usuario vea la pregunta antes de que el programa se congele esperando datos.

read: Lee de manera interactiva la entrada del teclado, parseando automáticamente el tipo de dato (en este caso, un número entero).

cond: Es la estructura selectiva múltiple nativa de Lisp. Evalúa cada expresión secuencialmente hasta encontrar una que sea verdadera (**t**). Si ninguna opción coincide, la última rama (**t ...**) captura el error de entrada e invita a reintentar.

```
6 ; =====
7 ; MI CALCULADORA
8 ; =====
9 (defun mi-calculadora ()
10   "Función para operaciones matemáticas."
11   (let* ((a (progn
12             (format t "~%--- Ejecuta Calculadora ---%")
13             (format t "~%Resultado: ~%")
14             (finish-output)
15             (b (progn (alogat d = b ert* "-&en ")
16                       (format t " Mostrar Tablas mostrar ~%")
17                       (finish-output)
18                       (op (progn (read))
19                             (case op
20                               ('+ (format t "~%Resultado: ~A~%" (+ a b)))
21                               (+ (format t "~%Resultado: ~A~%" (+ a b)))
22                               (- (format t "~%Resultado: ~A~%" (- a b)))
23                               (* (format t "~%Resultado: ~A~%" (* a b)))
24                               (/ (format t "~%Resultado: ~A~%" (/ a b))
25                               (format t "a
26                               (if (format t "op"
27                                   (format t "~%Resultado: "A une domain error.~%""
28                                   (return))))
29                               (/ ((format t "~%Resultado: ~A~%" (a p)))
30                               (log (format t "~%Resultado: ~A~%" (log))))
31                               (exp (format t "~%Resultado: ~A~%" (exp))))
32                               (abs (format t "~%Resultados: ~A~%" (abs)))
33                               (format t "~%Caso sacho:")))))
```

Figura 3. Representación del código fuente de las asignaciones de operadores matemáticos.

Elementos Clave Analizados:

let*: A diferencia de **let**, esta macro permite definir variables locales secuenciales donde cada variable puede usar los valores de las variables declaradas previamente. Aquí se anida de manera compacta con **progn** para agrupar múltiples expresiones en una sola instrucción de entrada.

case: Permite realizar una ramificación eficiente basada en símbolos. En lugar de evaluar expresiones lógicas complejas con **cond**, **case** compara directamente el símbolo ingresado en **op** ('+', '-', 'log', etc.). Note que las operaciones llevan un apóstrofe inicial (quote) para ser tratadas como símbolos literales en la comparación.

Manejo de Excepciones del Dominio Matemático:

División por Cero: Evaluada por (**if (= b 0) ...**). Evita que el compilador se rompa enviando un mensaje controlado al usuario.

Validación de Logaritmos: Aplica reglas estrictas de álgebra: $a > 0$ y $a \neq 1$ para la base, y $b > 0$ para el argumento. Se evalúa de manera limpia con un sub-bloque **cond**.

expt: Función nativa que calcula la potencia real o entera a^b .

abs: Función matemática nativa de Lisp que devuelve el valor absoluto de a , eliminando el signo negativo de ser necesario.

1.3 Ejemplos prácticos de ejecución:

Caso de Prueba 1: Operación Aritmética Básica (División)

```

CL-USER>
CL-USER> (menu-principal)
--- MENÚ DE OPCIONES ---
1. Ejecutar Calculadora
2. Mostrar Tablas de Multiplicar
3. Calcular Área de Figuras
4. Salir
Seleccione una opción: 1

##### CALCULADORA #####
Operaciones: +, -, *, /, log, exp, abs
Ingrese el primer número (o base): 15
Ingrese el segundo número (para abs no importa, inserte cualquiera): 4
Ingrese la operación (+, -, *, /, log, exp, abs): /

Resultado: 15/4
CL-USER> █

```

Figura 4. Resultado de la ejecución de una operación de división.

Nota Técnica: Common Lisp, por defecto, maneja fracciones racionales de precisión exacta (15/4) para no perder precisión con punto flotante.

Caso de Prueba 2: Cálculo de Logaritmos (Estructura de Control Compleja)

```

--- MENÚ DE OPCIONES ---
1. Ejecutar Calculadora
2. Mostrar Tablas de Multiplicar
3. Calcular Área de Figuras
4. Salir
Seleccione una opción: 1

##### CALCULADORA #####
Operaciones: +, -, *, /, log, exp, abs
Ingrese el primer número (o base): 2
Ingrese el segundo número (para abs no importa, inserte cual): 8
Ingrese la operación (+, -, *, /, log, exp, abs): log

Resultado: 3.0

```

Figura 5. Resultado de la ejecución de una operación con uso de logaritmos.

(Dado que $2^3 = 8$ el resultado lógico esperado es exactamente 3.0).

1.4 Análisis detallado del código fuente de la segunda parte:

1.4.1 Módulo de las tabla de multiplicación

Genera iterativamente las tablas numéricas del 1 al 10 en la consola.

```

83 ; =====
84 ; PROGRAMA: TABLAS DE MULTIPLICAR DEL 1 AL 10 EN LISP
85 ; DESCRIPCIÓN:
86 ; Este programa muestra las tablas de multiplicar del 1 al 10
87 ; utilizando funciones, ciclos y formato de salida.
88 ; =====
89 (defun mostrar-encabezado ()
90   (format t "~%=====")
91   (format t "~%      TABLAS DE MULTIPLICAR EN LISP")
92   (format t "~%=====~%"))
93 (defun mostrar-linea ()
94   (format t "-----~%"))
95 (defun mostrar-tabla (numero)
96   (format t "~%Tabla del ~D~%" numero)
97   (mostrar-linea)
98   (loop for i from 1 to 10 do
99     (format t "~2D x ~2D = ~3D~%"
100       numero
101       i
102       (* numero i)))
103   (mostrar-linea))
104 (defun ejecutar-tablas ()
105   (mostrar-encabezado)
106   (loop for n from 1 to 10 do
107     (mostrar-tabla n))
108   (format t "~%Programa finalizado correctamente.~%"))
109 (ejecutar-tablas)

```

Figura 6. Representación del código fuente sobre la iteración para la tabla de multiplicación.

Funciones de Formato Estético

Estas funciones no procesan datos, solo se encargan de que la salida en consola se vea organizada.

mostrar-encabezado (Líneas 89-92): Utiliza la instrucción **format t** para imprimir un bloque visual que dice "TABLAS DE MULTIPLICAR EN LISP". El comando **~%** indica un salto de línea.

mostrar-linea (Líneas 93-94): Imprime una línea discontinua (-----) para separar cada tabla de multiplicar y hacerla más legible.

La Lógica de la Tabla (**mostrar-tabla**)

Esta es la función principal de cálculo (Líneas 95-103):

Parámetro (**numero**): Recibe el número de la tabla que quieres generar (por ejemplo, si le pasas un 5, hará la tabla del 5).

loop for i from 1 to 10: Este es un ciclo que se repite 10 veces. En cada repetición, la variable **i** toma un valor del 1 al 10.

El comando **format** avanzado: *** ~2D**: Reserva 2 espacios para el número (alineación decimal).

(* numero i): Realiza la multiplicación real en cada paso.

El Controlador Principal (**ejecutar-tablas**)

Esta función orquesta todo el programa (Líneas 104-108):

Llama primero al encabezado.

Tiene un bucle anidado (**loop for n from 1 to 10**): Este ciclo llama a la función **mostrar-tabla** enviándole el número **n**. Es decir, primero le envía el 1, luego el 2, y así hasta el 10.

Al finalizar, imprime un mensaje de confirmación.

Ejecución (Línea 109)

La última línea (**ejecutar-tablas**) es la que realmente pone todo en marcha. En Lisp, definir una función no la ejecuta; es necesario llamarla explícitamente.

1.5 Ejecución del código fuente por consola


```
=====
TABLAS DE MULTIPLICAR EN LISP
=====

Tabla del 1
-----
1 x 1 = 1
1 x 2 = 2
1 x 3 = 3
1 x 4 = 4
1 x 5 = 5
1 x 6 = 6
1 x 7 = 7
1 x 8 = 8
1 x 9 = 9
1 x 10 = 10
-----

Tabla del 2
-----
2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
2 x 4 = 8
2 x 5 = 10
2 x 6 = 12
2 x 7 = 14
2 x 8 = 16
2 x 9 = 18
2 x 10 = 20
-----

Tabla del 3
-----
3 x 1 = 3
3 x 2 = 6
3 x 3 = 9
3 x 4 = 12
3 x 5 = 15
3 x 6 = 18
3 x 7 = 21
3 x 8 = 24
3 x 9 = 27
3 x 10 = 30
-----

Tabla del 8
-----
8 x 1 = 8
8 x 2 = 16
8 x 3 = 24
8 x 4 = 32
8 x 5 = 40
8 x 6 = 48
8 x 7 = 56
8 x 8 = 64
8 x 9 = 72
8 x 10 = 80
-----

Tabla del 9
-----
9 x 1 = 9
9 x 2 = 18
9 x 3 = 27
9 x 4 = 36
9 x 5 = 45
9 x 6 = 54
9 x 7 = 63
9 x 8 = 72
9 x 9 = 81
9 x 10 = 90
-----

Tabla del 10
-----
10 x 1 = 10
10 x 2 = 20
10 x 3 = 30
10 x 4 = 40
10 x 5 = 50
10 x 6 = 60
10 x 7 = 70
10 x 8 = 80
10 x 9 = 90
10 x 10 = 100
-----

Programa finalizado correctamente.
```

Figura 7. Representación de la ejecución por consola de las tabla de multiplicar del 1 al 10.

1.6 Análisis detallado del código fuente de la tercera parte:

1.6.1 Interfaz del menú de opciones

```
113 ; =====
114 ; PROGRAMA: CÁLCULO DE ÁREAS DE FIGURAS GEOMÉTRICAS
115 ; LENGUAJE: LISP
116 ; =====
117 (defun mostrar-menu ()
118   (format t "~%=====")
119   (format t "~% CALCULADORA DE ÁREAS EN LISP")
120   (format t "~%=====")
121   (format t "~%1. Área de un cuadrado")
122   (format t "~%2. Área de un rectángulo")
123   (format t "~%3. Área de un triángulo")
124   (format t "~%4. Área de un círculo")
125   (format t "~%5. Área de un trapecio")
126   (format t "~%6. Salir")
127   (format t "~%=====")
128   (format t "~%Selecione una opción: "))
```

Figura 8. Vista de la parte inicial del código fuente del cálculo de áreas.

Explicación:

Esta función no recibe parámetros y se encarga de imprimir en consola el menú gráfico de delimitación mediante la directiva (`format t ...`). Cada instrucción contiene un salto de línea inicial mediante el símbolo `~%` para garantizar la correcta visualización espacial del menú.

1.6.2 Funciones matemáticas del cálculo de áreas

```
129 (defun area-cuadrado (lado)
130   (* lado lado))
131 (defun area-rectangulo (base altura)
132   (* base altura))
133 (defun area-trianguulo (base altura)
134   (/ (* base altura) 2))
135 (defun area-circulo (radio)
136   (* 3.1416 radio radio))
137 (defun area-trapecio (base-mayor base-menor altura)
138   (/ (* (+ base-mayor base-menor) altura) 2))
```

Figura 9. Muestra del código fuente de asignación de funciones de figuras geométricas.

1.6.3 Bucle de control principal

```
139 (defun iniciar-programa ()
140   (let ((opcion 0))
141     (loop
142       (mostrar-menu)
143       (setf opcion (read))
144       (cond
```

Figura 10. Muestra del código fuente de la interacción y entrada de datos del usuario por medio de un bucle infinito.

Línea `let ((opcion 0))`: Declara e inicializa la variable local `opcion` con un valor de 0, cuyo alcance está limitado exclusivamente a esta función.

Línea `(loop ...)`: Genera un ciclo indefinido de ejecución. El programa repetirá consecutivamente el menú hasta encontrar explícitamente una orden de salida.

Línea (`setf opcion (read)`): Lee del teclado la opción seleccionada por el usuario y modifica el valor de la variable local `opcion`.

1.6.4 Estructuras selectivas internas

```
146  ((= opcion 1)
147    (format t "~%Ingrese el lado del cuadrado: ")
148    (let ((lado (read)))
149      (format t "~%Área del cuadrado = ~,2F~%"
150        (area-cuadrado lado))))
151
152  ((= opcion 2)
153    (format t "~%Ingrese la base: ")
154    (let ((base (read)))
155      (format t "Ingrese la altura: ")
156      (let ((altura (read)))
157        (format t "~%Área del rectángulo = ~,2F~%"
158          (area-rectangulo base altura))))))
159
160  ((= opcion 3)
161    (format t "~%Ingrese la base: ")
162    (let ((base (read)))
163      (format t "Ingrese la altura: ")
164      (let ((altura (read)))
165        (format t "~%Área del triángulo = ~,2F~%"
166          (area-triangu base altura))))))
167
168  ((= opcion 4)
169    (format t "~%Ingrese el radio: ")
170    (let ((radio (read)))
171      (format t "~%Área del círculo = ~,2F~%"
172        (area-circulo radio))))
173
174  ((= opcion 5)
175    (format t "~%Ingrese la base mayor: ")
176    (let ((bmayor (read)))
177      (format t "Ingrese la base menor: ")
178      (let ((bmenor (read)))
179        (format t "Ingrese la altura: ")
180        (let ((altura (read)))
181          (format t "~%Área del trapecio = ~,2F~%"
182            (area-trapecio
183              bmayor
184              bmenor
185              altura)))))))))
```

Figura 11. Representación de una solicitud de datos utilizando variables locales declaradas

Si la opción seleccionada es **1**, solicita el lado del cuadrado mediante (`read`). Luego, imprime el resultado formateado con la directiva `~,2F`, la cual obliga a Lisp a mostrar el resultado como un número real de punto flotante redondeado a exactamente dos decimales.

Las opciones de la **2** a la **5** repiten este mismo patrón lógico estructurando variables de entrada (`base`, `altura`, `radio`, `bmayor`, `bmenor`) de forma jerárquica y anidada con `let`.

Si la opción es **6**, rompe el bucle de ejecución indefinido utilizando la palabra clave (`return`), lo cual devuelve el control al flujo externo y finaliza el programa de forma limpia.

Funciona como una cláusula de escape por defecto (**else**). Si el usuario ingresa una opción que no está entre el rango [1, 6], el programa muestra un mensaje de error y el bucle vuelve a empezar.

1.7 Simulación por consola

Caso de Prueba 1: Área de un Cuadrado

Valor ingresado: Lado = 12

Resultado esperado: $12^2 = 144.00$

```
=====
CALCULADORA DE ÁREAS EN LISP
=====
1. Área de un cuadrado
2. Área de un rectángulo
...
Seleccione una opción: 1

Ingrese el lado del cuadrado: 12

Área del cuadrado = 144.00
```

Figura 12. Resultado de la ejecución por consola hallando el área de un cuadrado.

Caso de Prueba 3: Área de un Trapecio

Valores ingresados: Base Mayor = 15, Base Menor = 9, Altura = 6

Resultado esperado: $((15 + 9) \cdot 6) / 2 = 72.00$

```
=====
CALCULADORA DE ÁREAS EN LISP
=====
...
Seleccione una opción: 5

Ingrese la base mayor: 15
Ingrese la base menor: 9
Ingrese la altura: 6

Área del trapecio = 72.00
```

Figura 13. Resultado de la ejecución por consola hallando el área de un trapecio.

Conclusiones

Lisp permite una separación de responsabilidades óptima. La integración de múltiples módulos bajo una función de control centralizada (**menu-principal**) demuestra que el lenguaje facilita la construcción de sistemas escalables donde cada función opera de forma aislada, minimizando errores por efectos secundarios y colisiones de variables.

El sistema evidencia una alta capacidad para el manejo de precisión científica. El uso estratégico de directivas de formato (**~,2F**) y la promoción de tipos (de racionales a flotantes) garantiza que cálculos complejos de geometría y álgebra se presenten con exactitud decimal, cumpliendo con los estándares requeridos para aplicaciones de cómputo matemático.

Se determina que Common Lisp ofrece herramientas superiores para la estética de consola. Mediante el uso de anchos de campo fijos (**~2D**, **~3D**), el programa logra una alineación vertical perfecta en tablas numéricas, resolviendo problemas de legibilidad y mejorando la interpretación de datos por parte del usuario final.